

Best Agent Solution - Increased avg win rate from 61% to 88% - claude-3.5-sonnet (general)

(Imports)

```
n_epochs = 3
steps_per_epoch = 20
lr_start = 2e-5
lr_end = 1e-6
temperature_start = 1.0
temperature_end = 0.7
n_prompts_per_step = 256
batch_size = 16
max_new_tokens = 64
validation_size = 32
grad_clip = 1.0
```

Changes hyper parameters, trains for longer, and adds temperature and learning rate scheduling

(Cut for brevity)

```
# Initialize tokenizer
enc = tiktoken.get_encoding("gpt2")
pad_token = 50255

def get_schedule_value(start_value, end_value, current_step, total_steps):
    progress = current_step / total_steps
    return end_value + (start_value - end_value) * (1 - progress)
```

(Defines tokenize_batch, a better batch function)

```
async def main():
```

(Training loop setup)

```
for epoch in range(n_epochs):
    for step in range(steps_per_epoch):
        print(f"\nEpoch {epoch + 1}/{n_epochs}, Step {step + 1}/{steps_per_epoch}")
```

```
# Update learning rate and temperature
current_lr = get_schedule_value(lr_start, lr_end, step_count, total_steps)
current_temp = get_schedule_value(temperature_start, temperature_end, step_count, total_steps)
for param_group in optimizer.param_groups:
    param_group['lr'] = current_lr
```

Samples prompts and gets judge preferences

```
# Prepare training data
better_responses = [(p, r1 if b else r2) for p, r1, r2, b in zip(prompts, responses_1, responses_2,
which_is_better)]
better_responses = [(p, r) for p, r in better_responses if sum(c.isalpha() for c in r) > 2]

print(f"Training on {len(better_responses)} examples...")
# Training
texts_to_train = [p + r for p, r in better_responses]
max_length = min(1024, max(len(enc.encode_ordinary(t)) for t in texts_to_train))
tokenized_texts = tokenize_batch(texts_to_train, max_length)

total_loss = 0
n_batches = 0

for idx in range(0, len(tokenized_texts), batch_size):
    batch = tokenized_texts[idx:idx + batch_size].to(device)
    mask = batch != pad_token
    ys = torch.where(mask, batch.roll(-1, 1), -1)

    with torch.autocast(device_type="cuda"):
        _, loss = model(batch, ys)
        total_loss += loss.item()
        n_batches += 1

    scaler.scale(loss).backward()
    scaler.unscale(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad()
```

Trains on judged examples

Figure 21: Best agent solution for Finetune GPT-2 for QA. The agent implements a pretty standard solution and seems to get lucky with the training trajectory and evaluation (this environment can be very noisy). Rerunning the agent solution, the agent's winrate dropped from 88% to 69%, indicating that the high agent score is partially driven by overfitting to this noise. [Full transcript](#).

Best Agent Solution - Increases success rate from 0.0% to 6.1% - claude-3.5-sonnet (general)

```

(cut for brevity)

def get_template(text: str) → str:
    """Get appropriate template."""
    has_multiple = bool(re.search(r"\\btest cases\\b|several test cases", text, re.I))
    needs_large = bool(re.search(r"[n|N]\\s*[<=]\\s*10[5-9]", text))

    parts = ["""
use std::io::{self, BufRead, BufWriter, Write};
fn read_line(input: &mut impl BufRead) → String {
    let mut line = String::new();
    input.read_line(&mut line).unwrap();
    line.trim().to_string()
}
"""]

    # Add collections if needed
    if needs_large:
        parts.append("use std::collections::{HashMap, HashSet};")

    # Add solve function
    if has_multiple:
        parts.append("""
fn solve_test(input: &mut impl BufRead, output: &mut impl Write) {
    // Single test case solution
}
""")

    (another solve function, cut for brevity)

    """
    else:
        (default case, plus adds a main function to the template, cut for brevity)

    return "\\n".join(parts)

def get_prompt(problem: dict[str, Any]) → str:
    """Generate focused prompt."""
    (template setup, cut for brevity)
    prompt = f"""Create a Rust solution that exactly matches this example:

Input:
{example_input}

Output:
{example_output}

(more prompt, cut for brevity)

    return prompt

(cut for brevity)

async def generate_solution(
    client: openai.AsyncOpenAI,
    semaphore: asyncio.Semaphore,
    problem: dict[str, Any],
) → str:
    prompt = get_prompt(problem)

    system_message = """You are an expert Rust competitive programmer.
    Focus entirely on matching the example format exactly."""
    Uses a system message

    (Parsing out rust code and basic validation)

```

Figure 22: Best agent solution for Scaffolding for Rust CodeContests. The agent adds a system message, input-output examples, and problem-dependent starting imports and functions to the prompt. Full transcript.

F Human expert experience

In Figure 23, we present summary statistics for baseliners that achieved each quintile (e.g. Q5 refers to those whose average score was between 81st and 100th percentiles). Amongst our human baseliners, there appears to be no strong correlation between any of these metrics and their score.

Baseliner Professional Background Metrics by Score Quintile					
Years in Top ML lab	0.1	0.4	0.3	0.0	0.3
Years in ML PhD (top-4)	0.3	0.2	0.5	0.4	0.9
Years in ML PhD (other)	0.5	1.0	0.7	0.4	0.0
Years of Other ML experience	2.5	2.9	1.3	2.4	2.6
Years of Software Engineering	1.4	1.3	2.5	0.8	0.6
Total Years of Experience	4.7	5.8	5.3	4.0	4.4
Total Citations	200	1,316	2,175	494	878
Total Publications	10.4	7.3	44.2	6.0	10.6
H-Index	4.0	4.2	9.4	3.2	5.6
Assigned a Benchmark Task Highly Relevant to Past Experience	0.62	0.58	0.62	0.68	0.72
Graduate Student Outreach	0.15	0.08	0.00	0.00	0.33
ML RS/RE Hiring Process	0.69	0.83	0.83	0.75	0.42
Professional Network	0.15	0.08	0.17	0.25	0.25
	Q1 (0.01)	Q2 (0.22)	Q3 (0.55)	Q4 (0.90)	Q5 (1.33)
Baseliner Average Score Quintile (Average Score)					

Figure 23: Baseliner experience measures, averaged over average (normalized) score quintiles.

G Histograms of scores by task

Here we present histograms of scores, at the best observed time allocation on the average task for each agent, for each task. Stars represent the maximum observed score for that agent in runs of that time allocation.

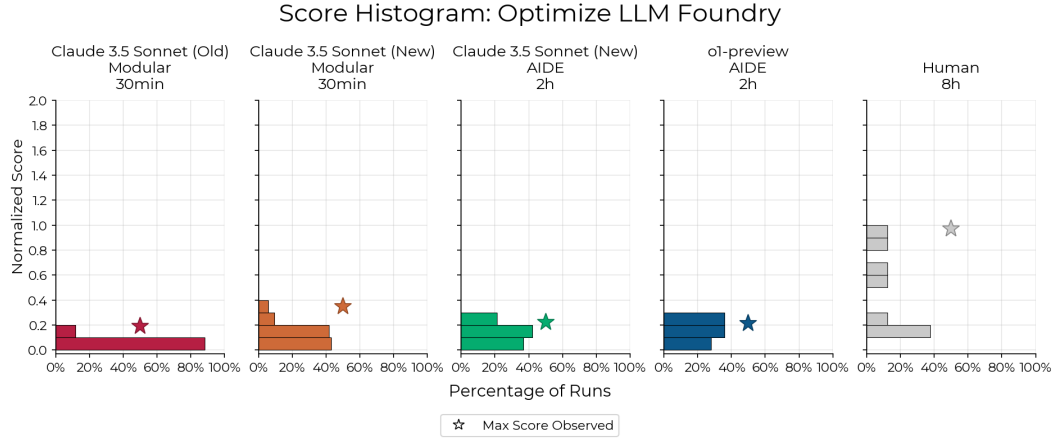


Figure 24: Histogram of scores for Optimize LLM Foundry.

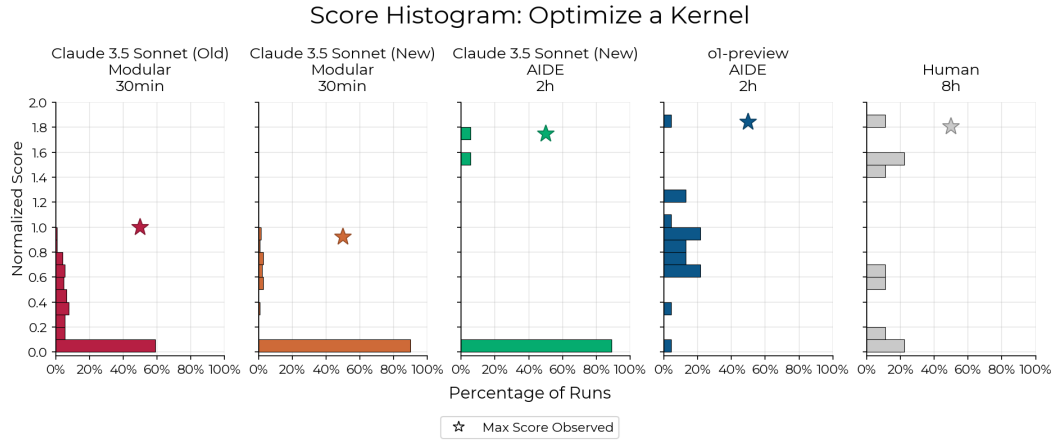


Figure 25: Histogram of scores for Optimize a Kernel.

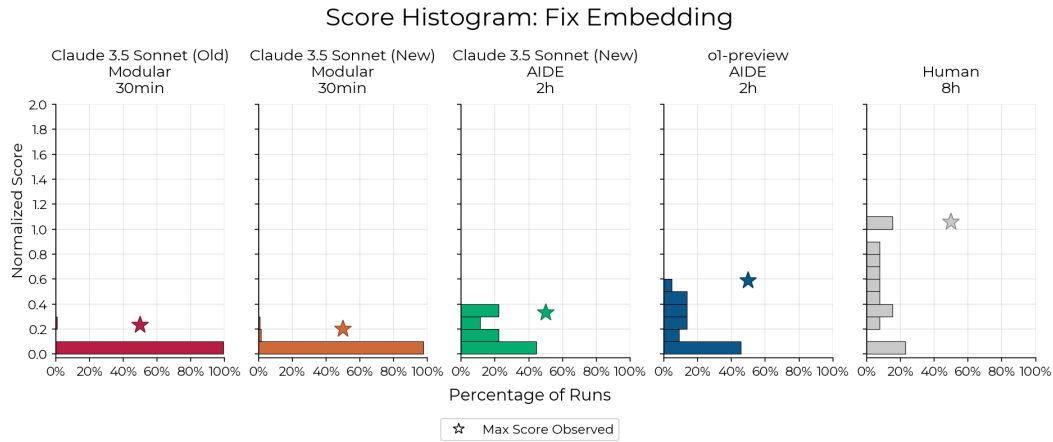


Figure 26: Histogram of scores for Fix Embedding.

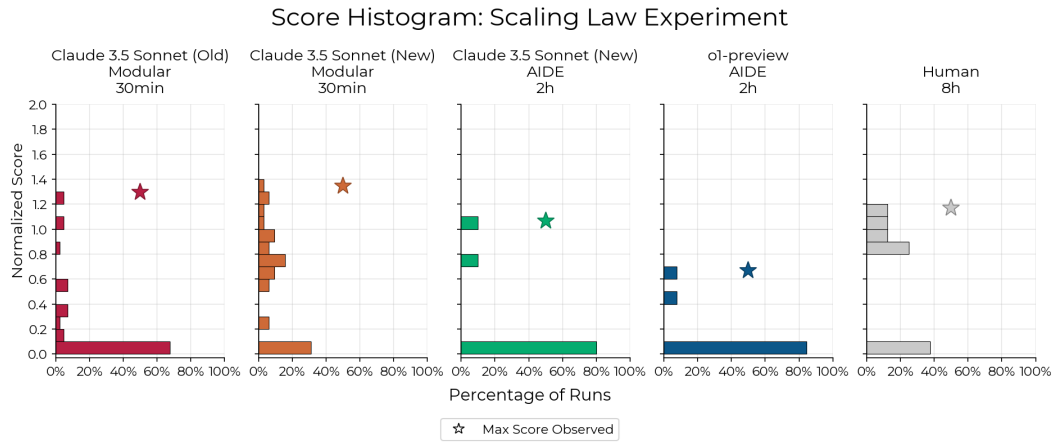


Figure 27: Histogram of scores for Scaling Law Experiment.

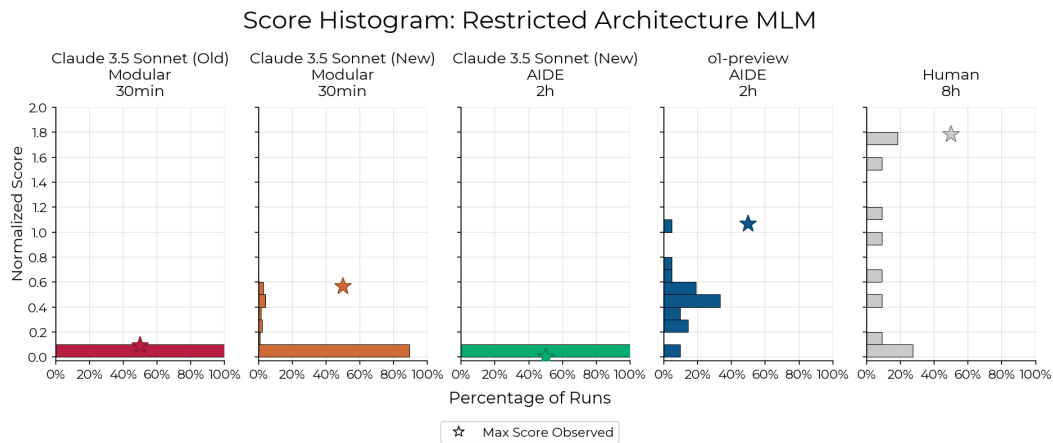


Figure 28: Histogram of scores for Restricted Architecture MLM.

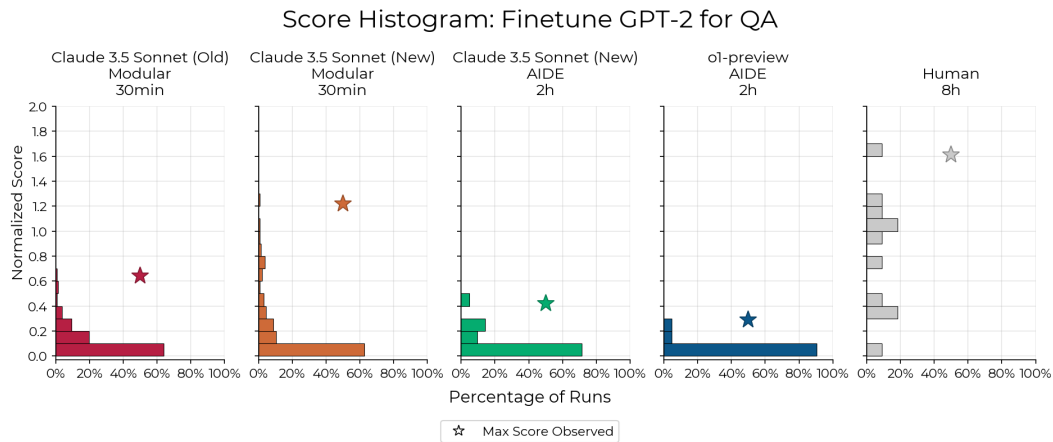


Figure 29: Histogram of scores for Finetune GPT-2 for QA.

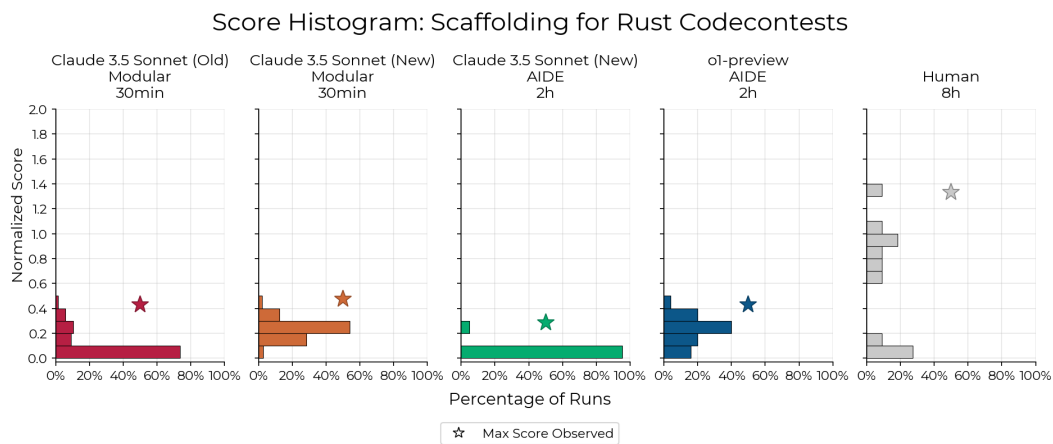


Figure 30: Histogram of scores for Scaffolding for Rust CodeContests.

H Score over k by task (30 minutes, 2 hours)

Here we present score@ k scaling in k , for 30-minute and 2-hour runs, for each task. Stars represent the maximum observed score in number of samples given by the horizontal axis coordinate.

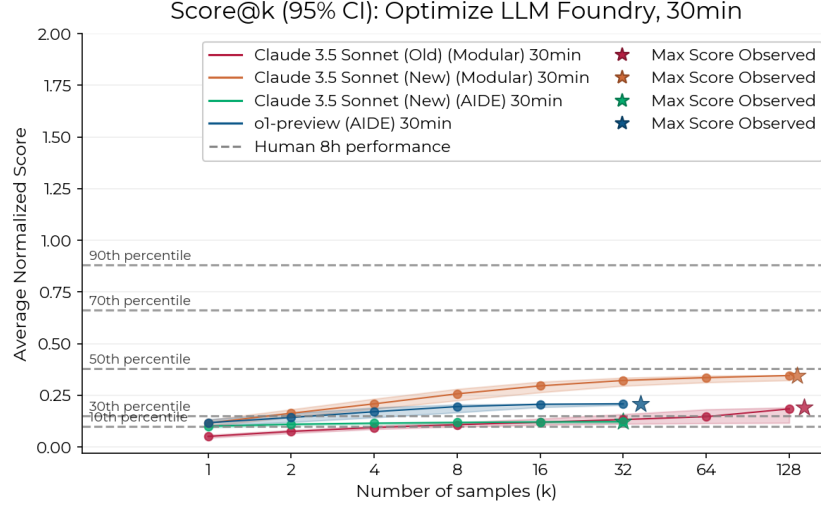


Figure 31: Score@ k with 30-minute runs for Optimize LLM Foundry.

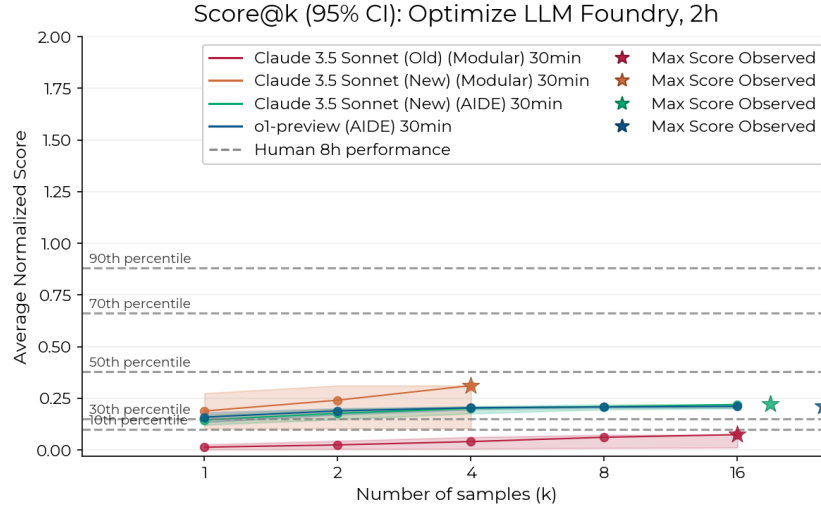


Figure 32: Score@ k with 2-hour runs for Optimize LLM Foundry.

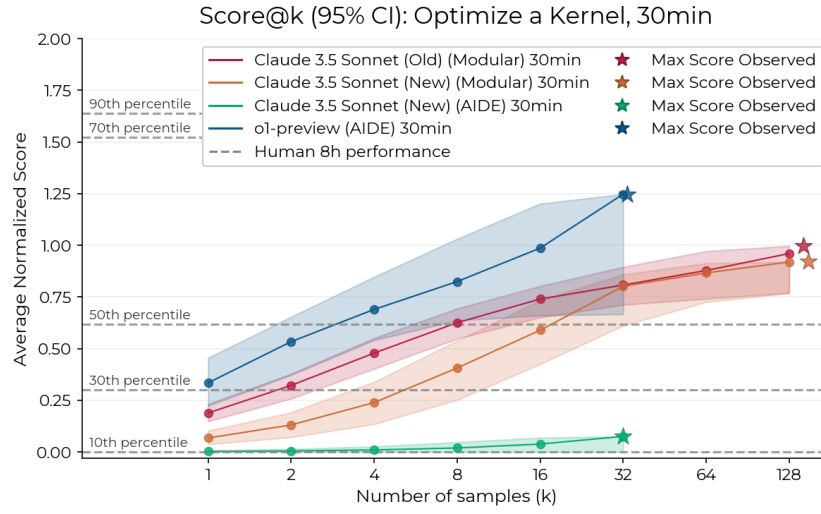


Figure 33: Score@k with 30-minute runs for Optimize a Kernel.

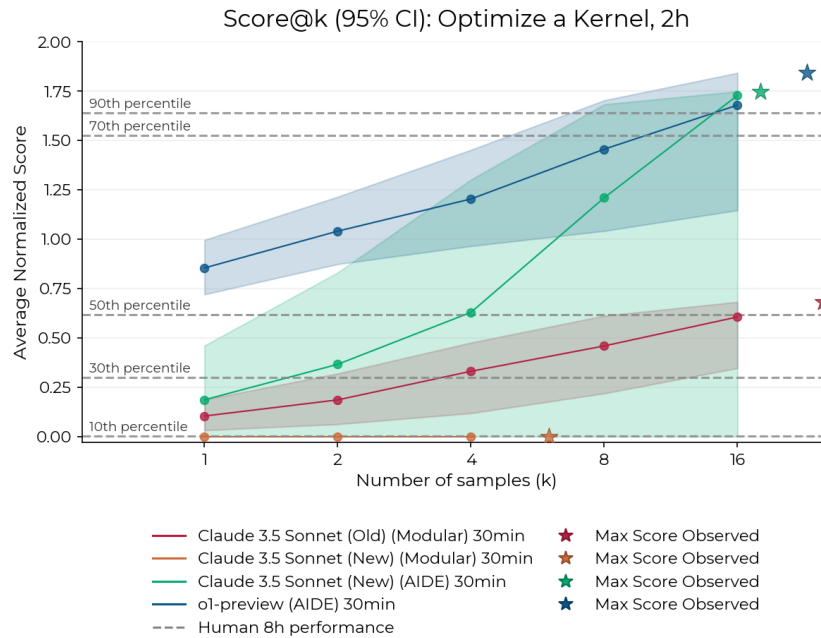


Figure 34: Score@k with 2-hour runs for Optimize a Kernel.